

Module 3 — Docker: Exercises

Exercise 1 — Prove it: containers share the kernel

```
Problems Output Debug Console Terminal Ports
@Phoptorn →/workspaces/DPU-Docker-Basic (main) $ uname -r
6.8.0-1064-azure
@Phoptorn →/workspaces/DPU-Docker-Basic (main) $ docker run --rm python:3.11 uname -r
Unable to find image 'python:3.11' locally
3.11: Pulling from library/python
762734b8a939: Pull complete
428ecea118d3: Pull complete
42f0cc32f2e3: Pull complete
e7272a918094: Pull complete
6eefb2f5d3e9: Pull complete
38985a14f2b1: Pull complete
55aabd477835: Pull complete
c983dda78ff7: Download complete
db429c597811: Download complete
Digest: sha256:70e8e937c1da72df1688e883a2108b0feff03381187fa78eca6d9b10df411320
Status: Downloaded newer image for python:3.11
6.8.0-1064-azure
@Phoptorn →/workspaces/DPU-Docker-Basic (main) $ time docker run --rm python:3.11 echo "hi"
hi

real    0m0.439s
user    0m0.031s
sys     0m0.018s
@Phoptorn →/workspaces/DPU-Docker-Basic (main) $
```

Question: Does the container's kernel version match your Codespace's? How many seconds did the timed run take?

> Yes, The container kernel versions match. time run
0m0.367s.

Exercise 2 — Run your first container

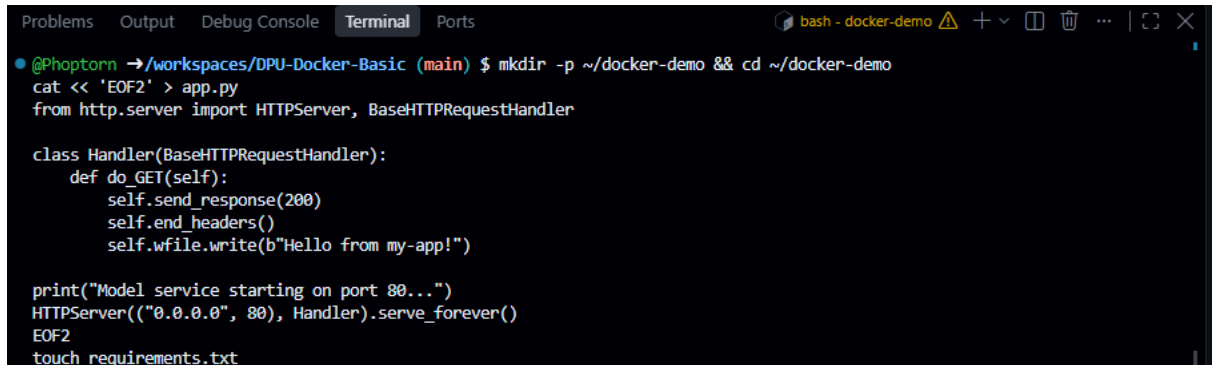
```
Problems Output Debug Console Terminal Ports
@Phoptorn →/workspaces/DPU-Docker-Basic (main) $ docker run -it python:3.11
Python 3.11.16 (main, Sep 19 2026, 03:37:02) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("hello from inside a container")
hello from inside a container
>>> exit()
@Phoptorn →/workspaces/DPU-Docker-Basic (main) $ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS      PORTS      NAMES
8a27583f9c11   python:3.11 "python3"               46 seconds ago Exited (0) 13 seconds ago      boring_raman
@Phoptorn →/workspaces/DPU-Docker-Basic (main) $
```

Question: Is the container still listed after you exit? What status does it show?

> It's still show and status exited.

Exercise 3 — Build your first image (a real, running service)

1. Create the app. This one actually starts a tiny web server — using only Python's standard library, so there's nothing to install:



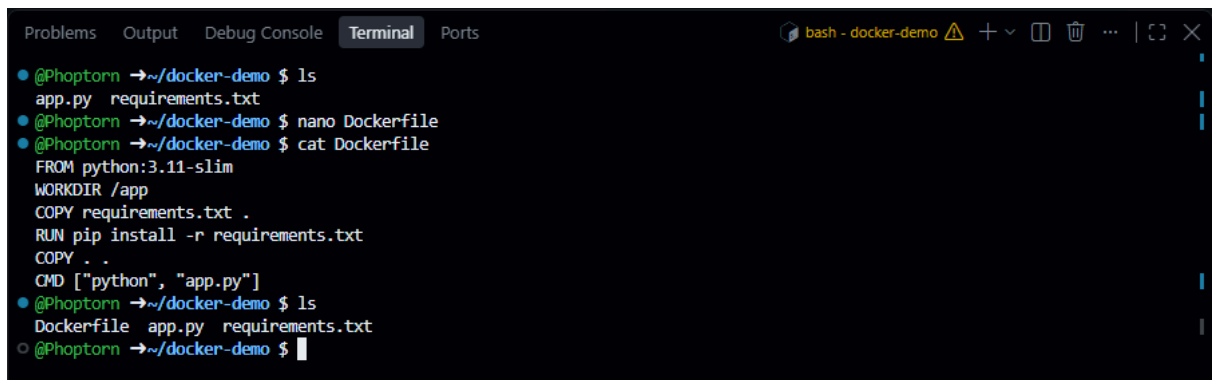
```
Problems Output Debug Console Terminal Ports
bash - docker-demo

@Phoptorn →/workspaces/DPU-Docker-Basic (main) $ mkdir -p ~/docker-demo && cd ~/docker-demo
cat << 'EOF2' > app.py
from http.server import HTTPServer, BaseHTTPRequestHandler

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"Hello from my-app!")

print("Model service starting on port 80...")
HTTPServer(("0.0.0.0", 80), Handler).serve_forever()
EOF2
touch requirements.txt
```

2. Create a file named Dockerfile (no extension) with this content:



```
Problems Output Debug Console Terminal Ports
bash - docker-demo

@Phoptorn →~/docker-demo $ ls
app.py  requirements.txt
@Phoptorn →~/docker-demo $ nano Dockerfile
@Phoptorn →~/docker-demo $ cat Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
@Phoptorn →~/docker-demo $ ls
Dockerfile app.py requirements.txt
@Phoptorn →~/docker-demo $
```

3. Build and check the image:

```
Problems Output Debug Console Terminal Ports bash - docker-demo + - - - - - X
@Phoptorn → ~/docker-demo $ docker build -t my-app .
[+] Building 12.5s (11/11) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 168B 0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 2.4s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:da047cb8f9d1d98e5c070f5300ba9f7274e33b8fc0e5be5ed88740aed1b9 3.8s
=> => resolve docker.io/library/python:3.11-slim@sha256:da047cb8f9d1d98e5c070f5300ba9f7274e33b8fc0e5be5ed88740aed1b9 0.0s
=> => sha256:0526d5e29bf341cb1941d46a6d4ab64fcd4fca3dc3adccfb4dbf5728b88b61e 249B / 249B 0.3s
=> => sha256:58abdd9670ca8c69d03432ce188995dee67f0979357caf83866fe1ef2ec37138 14.45MB / 14.45MB 1.6s
=> => sha256:41e7217c2e506d048f89ca0c5cfe848c05ef990900627c19f99938c4aae57f8 1.29MB / 1.29MB 1.6s
=> => sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db0d81151570244c8 29.83MB / 29.83MB 1.6s
=> => extracting sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db0d81151570244c8 1.1s
=> => extracting sha256:41e7217c2e506d048f89ca0c5cfe848c05ef990900627c19f99938c4aae57f8 0.1s
=> => extracting sha256:58abdd9670ca8c69d03432ce188995dee67f0979357caf83866fe1ef2ec37138 0.7s
=> => extracting sha256:0526d5e29bf341cb1941d46a6d4ab64fcd4fca3dc3adccfb4dbf5728b88b61e 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 570B 0.0s
=> [2/5] WORKDIR /app 2.8s
=> [3/5] COPY requirements.txt . 0.1s
=> [4/5] RUN pip install -r requirements.txt 2.3s
=> [5/5] COPY . . 0.1s
=> exporting to image 1.0s
=> => exporting layers 0.6s
=> => exporting manifest sha256:1f4103144e3362b2010c96d9c94544764d32422098c4c23909d2007c30f9079 0.0s
=> => exporting config sha256:848a9dad2572327a5fffd53f32f9cb35f44804047c026907be8cb17277f525c9d 0.0s
=> => exporting attestation manifest sha256:aab65460750957cf7bc74a3b9b923e551630b4fb63ac5bf4eb6500b52dc92dd4 0.0s
=> => exporting manifest list sha256:9bc568f76438bf37fa9844c52d0bb55aa6b92efa531dacb9d67629234f686f2 0.0s
=> => naming to docker.io/library/my-app:latest 0.0s
=> => unpacking to docker.io/library/my-app:latest 0.3s
@Phoptorn → ~/docker-demo $ docker images
INFO → U In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
my-app:latest 9bc568f76438 204MB 49.8MB
python:3.11 70e8e937c1da 1.61GB 427MB U
@Phoptorn → ~/docker-demo $
```

Question: Do you see my-app listed in docker images?

> Yes, I can see my-app in docker images.

Exercise 4 — Mount your dataset folder

```
Problems Output Debug Console Terminal Ports bash - docker-demo + - - - - - X
@Phoptorn → ~/docker-demo $ mkdir -p ~/docker-demo/data
echo "sample,row" > ~/docker-demo/data/sample.csv
cd ~/docker-demo
docker run -v ./data:/app/data -it python:3.11 bash
root@d003ee3474af:/# ls /app/data
cat /app/data/sample.csv
exit
sample.csv
sample,row
exit
@Phoptorn → ~/docker-demo $
```

Question: Do your host files show up inside the container?

> Yes. The host files show up inside the container.

Exercise 5 — Reach your container from outside

Uses the my-app image you built in Exercise 3 — it's already listening on port 80.

```
Problems Output Debug Console Terminal Ports
@Phoptorn → ~/docker-demo $ docker run -d -p 8000:80 --name my-app-container my-app
curl localhost:8000
f2c29e0f84f54153dc3ed67f840f2285a86294aff5a94ab4a99a34c412543f73
Hello from my-app!@Phoptorn → ~/docker-demo $
```

Question: Does curl get a response back? Try docker logs my-app-container if not.

> Yes, It's a response back from curl command, this is output Hello from my-app!

```
@Phoptorn → ~/docker-demo $ docker logs my-app-container
172.17.0.1 - - [25/Sep/2026 04:43:51] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [25/Sep/2026 04:43:58] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [25/Sep/2026 04:43:58] "GET /favicon.ico HTTP/1.1" 200 -
@Phoptorn → ~/docker-demo $
```

Exercise 6 — Bring up two services with Compose

1. From the same ~/docker-demo folder, create docker-compose.yml:

```
Problems Output Debug Console Terminal Ports
@Phoptorn → ~/docker-demo $ nano docker-compose.yml
@Phoptorn → ~/docker-demo $ cat docker-compose.yml
services:
  api:
    build: .
    ports:
      - "8000:80"
  redis:
    image: redis:7
```

2. Start everything:

```
@Phoptorn → ~/docker-demo $ docker compose up
=> => transferring dockerfile: 168B 0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.3s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:da047cb8f9d1d98e5c070f5300ba9f7274e33b8fc0e5be5ed88740aed1b9 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:da047cb8f9d1d98e5c070f5300ba9f7274e33b8fc0e5be5ed88740aed1b9 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 304B 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY requirements.txt . 0.0s
=> CACHED [4/5] RUN pip install -r requirements.txt 0.0s
=> [5/5] COPY . . 2.3s
=> exporting to image 0.2s
=> => exporting layers 0.1s
=> => exporting manifest sha256:d19ab4d59f56014808411765480694f1eb9dc59cda2fc599bda9cfd22973ee1b 0.0s
=> => exporting config sha256:aba87deac9fb002b990d79187ef2c5d7c72dced98bfefd7b0551c5ead47b6713 0.0s
=> => exporting attestation manifest sha256:f7c0f5844c1307af8d43140358b9eee00f62c91ddb339b5bfa3230bf51578073 0.0s
=> => exporting manifest list sha256:a347180943fb2696b1f65579caeb7415af219a91c11de62244e9fa0f72f6c3e5 0.0s
=> => naming to docker.io/library/docker-demo-api:latest 0.0s
[+] up 14/14king to docker.io/library/docker-demo-api:latest 0.0s
✓ Image redis:7 Pulled 5.4s
✓ Image docker-demo-api Built 4.2s
✓ Network docker-demo_default Created 0.1s
✓ Container docker-demo-redis-1 Created 0.1s
✓ Container docker-demo-api-1 Created 0.1s
Attaching to api-1, redis-1
redis-1 | 1:C 25 Sep 2026 04:49:34.736 # WARNING Memory overcommit must be enabled! Without it, a background save or replica
tion may fail under low memory condition. Being disabled, it can also cause failures without low memory condition, see http
s://github.com/jemalloc/jemalloc/issues/1328. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.conf and then
reboot or run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
redis-1 | 1:C 25 Sep 2026 04:49:34.736 * o000o000o000o Redis is starting o000o000o000o
redis-1 | 1:C 25 Sep 2026 04:49:34.736 * Redis version=7.4.11, bits=64, commit=00000000, modified=0, pid=1, just started
redis-1 | 1:C 25 Sep 2026 04:49:34.736 # Warning: no config file specified, using the default config. In order to specify a
config file use redis-server /path/to/redis.conf
redis-1 | 1:M 25 Sep 2026 04:49:34.737 * Increased maximum number of open files to 10032 (it was originally set to 1024).
redis-1 | 1:M 25 Sep 2026 04:49:34.737 * monotonic clock: POSIX clock_gettime
redis-1 | 1:M 25 Sep 2026 04:49:34.739 * Running mode=standalone, port=6379.
redis-1 | 1:M 25 Sep 2026 04:49:34.739 * Server initialized
redis-1 | 1:M 25 Sep 2026 04:49:34.739 * Ready to accept connections tcp
api-1 | 172.18.0.1 - - [25/Sep/2026 04:49:38] "GET / HTTP/1.1" 200 -
api-1 | 172.18.0.1 - - [25/Sep/2026 04:49:39] "GET /favicon.ico HTTP/1.1" 200 -
api-1 | 172.18.0.1 - - [25/Sep/2026 04:49:59] "GET / HTTP/1.1" 200 -
api-1 | 172.18.0.1 - - [25/Sep/2026 04:49:59] "GET /favicon.ico HTTP/1.1" 200 -
```

3. In a second terminal:

```
Problems Output Debug Console Terminal Ports ⓘ
● @Phoptorn → /workspaces/DPU-Docker-Basic (main) $ curl localhost:8000
○ Hello from my-app!@Phoptorn → /workspaces/DPU-Docker-Basic (main) $
```

Question: In the logs, can you see both the api and redis services starting? Stop everything with Ctrl+C then docker compose down.

> I see both the api and redis in the terminal.

```
✓ Container docker-demo-redis-1 Created
0.1s
✓ Container docker-demo-api-1 Created
0.1s
Attaching to api-1, redis-1
redis-1 | 1:C 25 Sep 2026 04:49:34.736 # WARNING Memory overcommit must be enabled! Without it, a background save or replication may fail under low memory condition. Being disabled, it can also cause failures without low memory condition, see https://github.com/jemalloc/jemalloc/issues/1328. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.conf and then reboot or run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
redis-1 | 1:C 25 Sep 2026 04:49:34.736 * o000o000o000o Redis is starting o000o000o000o
redis-1 | 1:C 25 Sep 2026 04:49:34.736 * Redis version=7.4.11, bits=64, commit=00000000, modified=0, pid=1, just started
redis-1 | 1:C 25 Sep 2026 04:49:34.736 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
redis-1 | 1:M 25 Sep 2026 04:49:34.737 * Increased maximum number of open files to 10032 (it was originally set to 1024).
redis-1 | 1:M 25 Sep 2026 04:49:34.737 * monotonic clock: POSIX clock_gettime
redis-1 | 1:M 25 Sep 2026 04:49:34.739 * Running mode=standalone, port=6379.
redis-1 | 1:M 25 Sep 2026 04:49:34.739 * Server initialized
redis-1 | 1:M 25 Sep 2026 04:49:34.739 * Ready to accept connections tcp
api-1 | 172.18.0.1 - - [25/Sep/2026 04:49:38] "GET / HTTP/1.1" 200 -
api-1 | 172.18.0.1 - - [25/Sep/2026 04:49:39] "GET /favicon.ico HTTP/1.1" 200 -
api-1 | 172.18.0.1 - - [25/Sep/2026 04:49:59] "GET / HTTP/1.1" 200 -
api-1 | 172.18.0.1 - - [25/Sep/2026 04:49:59] "GET /favicon.ico HTTP/1.1" 200 -
api-1 | 172.18.0.1 - - [25/Sep/2026 04:52:29] "GET / HTTP/1.1" 200 -
api-1 | 172.18.0.1 - - [25/Sep/2026 04:52:47] "GET / HTTP/1.1" 200 -
```